This is the function we will use to choose all the actions for each agent after we get an observation. It utilizes a mean field Q function parameterized by `param` and `layers` to sample actions for all agents using the softmax policy. The `infer_acts` function takes the following parameters (vector dimensions in parentheses for each):

- `obs` is the observation tensor, $N \times 13 \times 13 \times 2$.
- `mean_fields` is tensor containing all the mean field actions for each agent, $N \times 21$.
- `pos_list` is list of positions for each agent returned by `env.get_pos(...)`.
- `acts` is a vector of integer-represented actions of each agent (N,).
- `num_iter` is the number of times to alternative between action sampling and policy updates.
- `temp` is the softmax policy temperature, to control the exploration rate.

The function returns a tuple:

- `acts_` is a vector of integer actions sampled from policy (N,).
- `mean_fields_` is a tensor of mean field vectors for each agent (N,21).
- `qvals` is a tensor of Q values for each action for each agent (N,21).

Lastly, we need the function that does the training. We will give this function our parameter vector and experience replay buffer and let it do the mini-batch stochastic gradient descent.

Listing 9.18. The training function

```
def train(batch_size,replay,param,layers,J=64,gamma=0.5,lr=0.001):
    ids = np.random.randint(low=0,high=len(replay),size=batch_size)
1
    exps = [replay[idx] for idx in ids]
2
    losses = []
    jobs = torch.stack([ex[0] for ex in exps]).detach()
3
    jacts = torch.stack([ex[1] for ex in exps]).detach()
4
```